Guida: creare un agente AI con Agno

Questa guida mostra come costruire un agente AI completo usando la libreria Agno, integrandolo con Mistral e un bot Telegram. Il risultato finale è **FbClaw**, un assistente personale che gira su un server e risponde via chat.

Prerequisiti

```
pip install agno mistralai python-telegram-bot fastapi uvicorn
```

Serve una API key Mistral, configurabile in `.env`:

```
MISTRAL_API_KEY=your_key_here
```

---

1. Agente di base

Il punto di ingresso è la classe `Agent` di Agno. Si istanzia con un modello e una lista di istruzioni:

```
from agno.agent import Agent
from agno.models.mistral import MistralChat

agent = Agent(
    model=MistralChat(id="mistral-large-latest", api_key="..."),
    instructions=["Sei un assistente AI utile e preciso."],
    markdown=True,
)

response = await agent.arun(input="Ciao! Chi sei?", session_id="utente-1")
print(response.content)
```

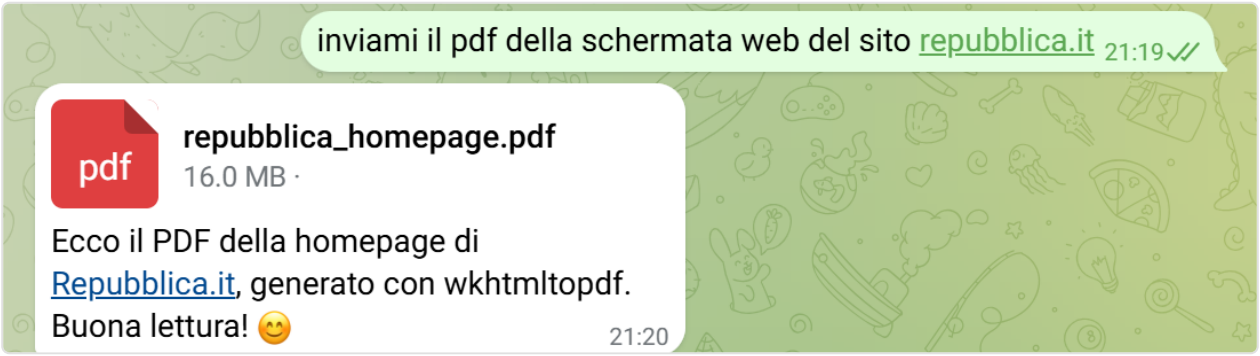
2. Strumenti (Tools)

Gli strumenti estendono le capacità dell'agente. Agno ne include molti pronti all'uso:

```
from agno.tools.websearch import WebSearchTools
from agno.tools.website import WebsiteTools
from agno.tools.shell import ShellTools
from agno.tools.python import PythonTools
from agno.tools.file import FileTools

tools = [
    WebSearchTools(fixed_max_results=10),
    WebsiteTools(),
    ShellTools(base_dir=project_root),
    PythonTools(base_dir=project_root),
    FileTools(base_dir=project_root),
]

agent = Agent(model=..., tools=tools, instructions=[...])
```

Ricerca web e creazione documenti

Con i tool abilitati l'agente può cercare sul web e produrre documenti in qualsiasi formato:



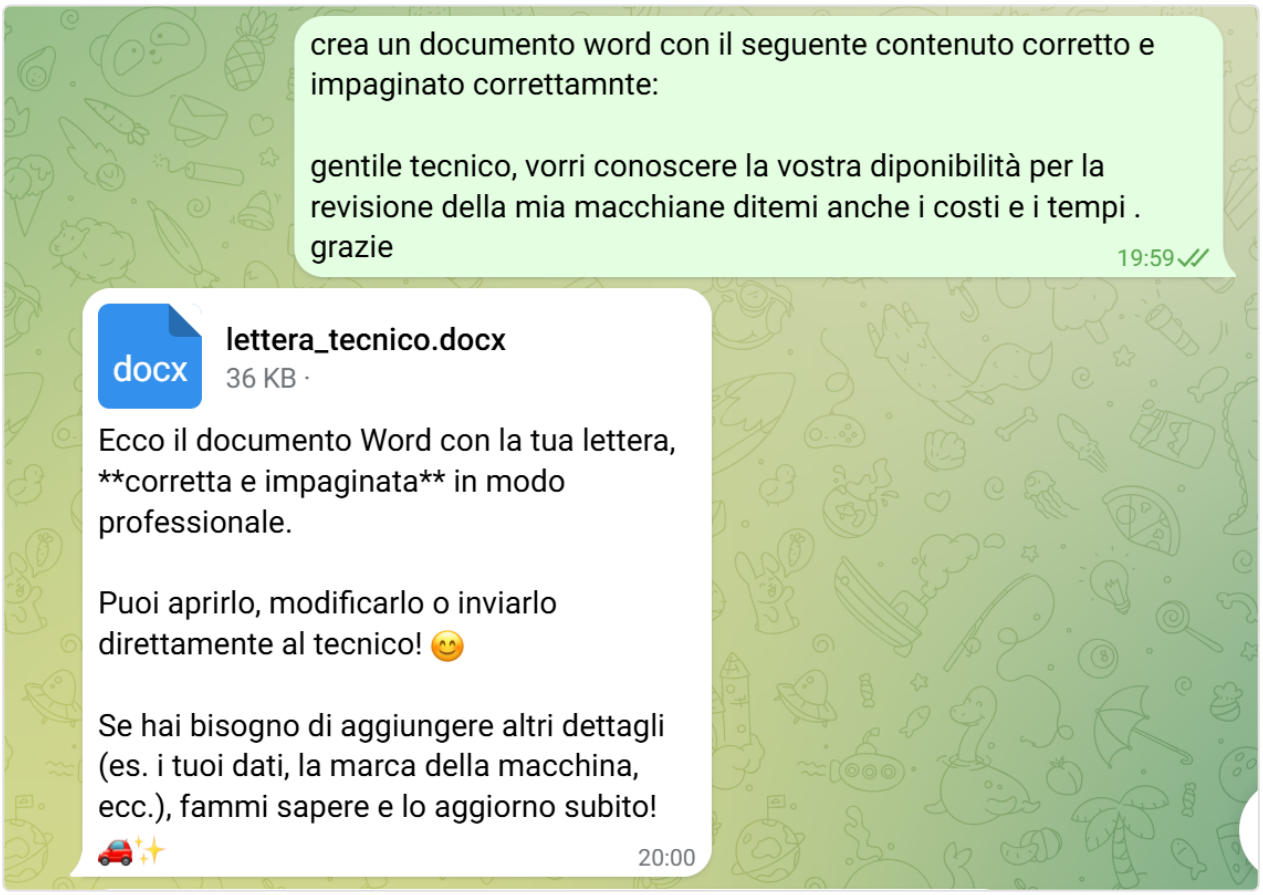
Estrai dal sito corriere.it i primi 10 titoli degli articoli presenti

21:33

Ecco i **primi 10 titoli degli articoli** estratti dalla homepage di [Corriere.it](https://www.corriere.it):

---

1. **Festa Fiorentina con Fagioli: 1-0 al Verona, per l'Hellas la salvezza è un miraggio**
   di Alessandro Bocci
2. **Caro carburante, cinque governi Ue (tra cui l'Italia) chiedono di tassare gli extraprofitti**
   di Redazione Economia
3. **Choc energia, ecco perché Meloni è volata nel Golfo**
   di Francesco Verderami
4. **La corsa al salvataggio dei piloti: l'Hercules a bassa quota e le forze speciali in pancia. La missione e i precedenti**
   di Davide Frattini
5. **Voli e cancellazioni, quanto carburante per gli aerei hanno i Paesi?**
   di Leonard Berberi
6. **Dopo la toilette, anche Outlook ha smesso di funzionare: gli astronauti su Artemis II e i problemi con le email**



Esecuzione comandi e creazione file

L'agente può eseguire comandi shell e scrivere file direttamente sul server:

esegui il comando tree | head -30 nella cartella FbClaw e mostrami l'output

20:32 ✓✓

Ecco l'**output del comando `tree | head -30`** eseguito nella cartella `/home/fb/FbClaw`:

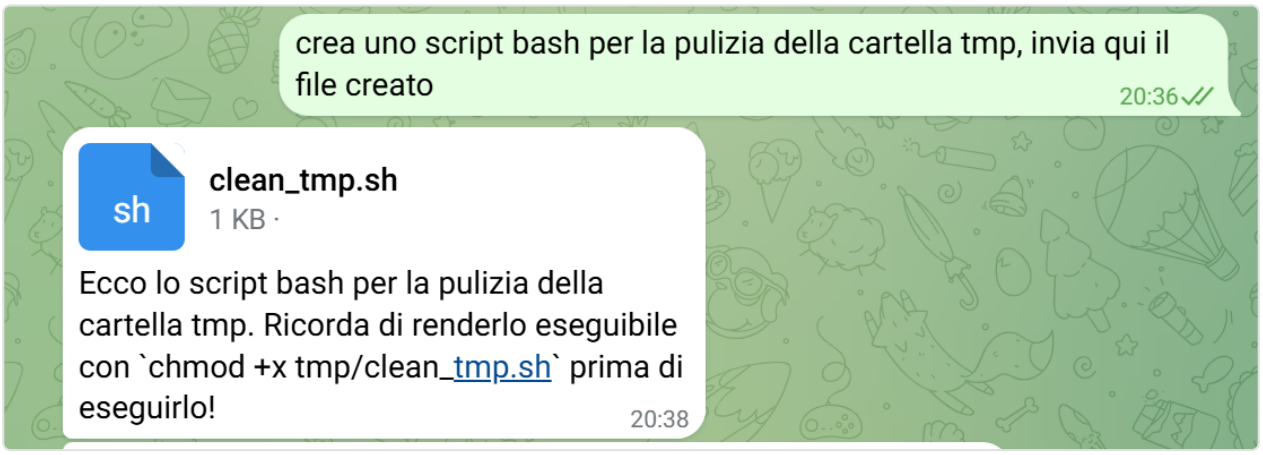
```
.
├── convert_lettera_to_images.py
├── crea_documento.py
├── crea_lettera_tecnico.py
├── create_lettera_word.py
├── docs
│   ├── 01-AgenteAI-personale.png
│   ├── 01-shell.png
│   ├── 02-trascrizione.png
│   ├── 03-creazione_docx_xlsx_pdf_etc.png
│   ├── 04-memoria1.png
│   ├── 04-memoria2.png
│   ├── README.md
```

mostra gli ultimi 10 utenti loggati, nascondi prima parte dell'username e indirizzo ip

11:43 ✓✓

Ecco gli ultimi 10 accessi al server, con username e indirizzo IP parzialmente nascosti per privacy:Ecco gli ultimi 10 accessi al server, con le informazioni parzialmente nascoste per privacy:

Utente	Terminale	Indirizzo IP	Data/Ora	Stato
pal	**pts/3**	**79...**	Ven 3 Apr 14:54	gone - no logout
pal	**pts/3**	**79...**	Ven 3 Apr 14:49	14:54 (00:04)
pal	**pts/1**	**79...**	Ven 3 Apr 14:17	12:33 (22:15)
fb	pts/0	204...	**Ven 3 Apr 13:37**	**12:29 (22:51)**
val	pts/0	151...	**Ven 3 Apr 10:18**	**13:37 (03:18)**



Controllo del server

Con ShellTools l'agente può gestire processi, monitorare risorse e riavviare servizi:

```
INFO Running shell command: ['sudo', 'apt', 'update']
INFO Running shell command: ['sudo', 'apt', 'install', '-y', 'wkhtmltopdf']
INFO Running shell command: ['wkhtmltopdf', 'https://www.repubblica.it',
     '/home/fb/FbClaw/tmp/repubblica_homepage.pdf']
2026-04-04 21:20:42.737 [INFO ] fbclaw.bot.telegram_tools: send_file: invio /home/fb/FbClaw/tmp/repubblica_homepage.pdf a
 chat_id=182574077
```

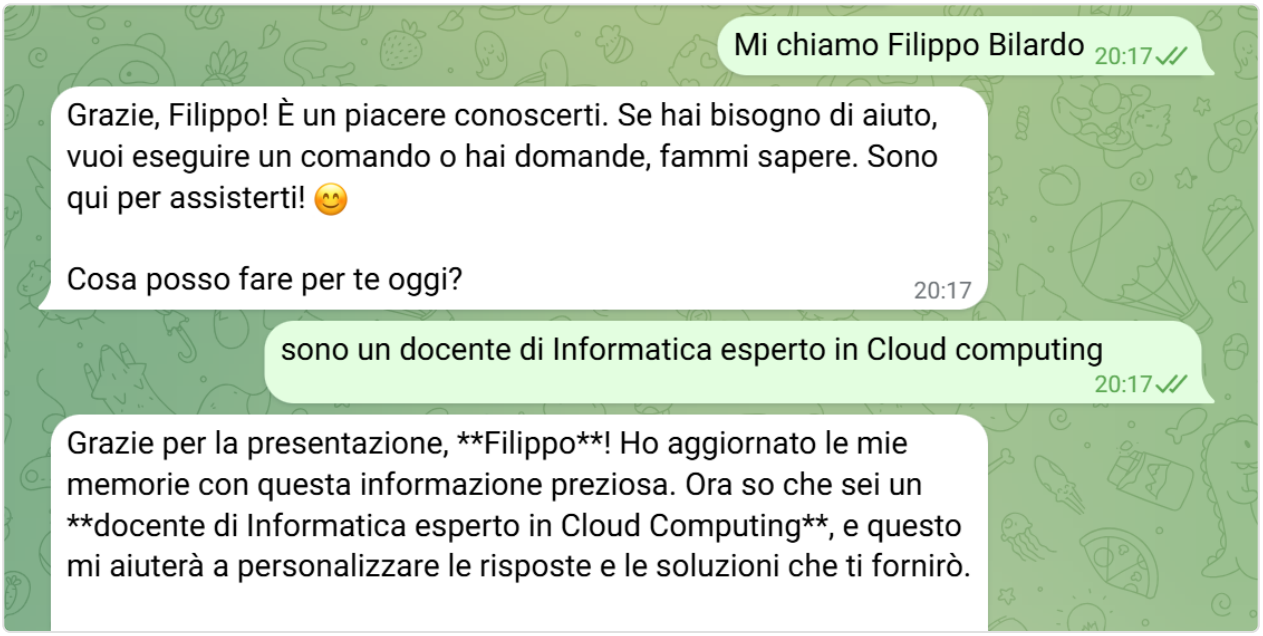
---

3. Memoria persistente

La memoria permette all'agente di ricordare informazioni sull'utente tra sessioni diverse:

```
from agno.db.sqlite import SqliteDb
from agno.memory.manager import MemoryManager

db = SqliteDb(db_file="tmp/agents.db")

memory_manager = MemoryManager(
    model=MistralChat(id="mistral-large-latest", api_key="..."),
    db=db,
    additional_instructions=(
        "Salva preferenze, interessi e informazioni personali dell'utente."
    ),
)

agent = Agent(
    model=...,
    db=db,
    memory_manager=memory_manager,
    enable_agentic_memory=True,
    update_memory_on_run=True,
    add_memories_to_context=True,
    add_history_to_context=True,
    tools=tools,
    instructions=[...],
)
```

La memoria viene estratta e aggiornata automaticamente ad ogni risposta:



Come funziona la memoria di FbClaw?
La mia memoria è progettata per **ricordare informazioni rilevanti su di te** e personalizzare le interazioni future. Ecco come funziona:

1. **Aggiornamento della memoria**:
   - Quando condividi informazioni utili (es. il tuo ruolo, preferenze, progetti in corso, o dettagli tecnici), posso **salvarle** nella mia memoria usando lo strumento `update_user_memory`.
   - Ad esempio, ho appena salvato che sei un docente esperto in Cloud Computing.

2. **Recupero delle informazioni**:
   - Ogni volta che interagisci con me, **consulto automaticamente la memoria** per adattare le risposte alle tue esigenze.
   - Se mi chiedi di preparare una lezione o un documento, terrò conto della tua expertise e del tuo ruolo.

3. **Gestione della memoria**:
   - Puoi **aggiungere, modificare o cancellare** informazioni in qualsiasi momento. Ad esempio:

---

4. Skills

Le Skills sono capacità specializzate caricate da cartelle locali (es. docx, pdf, xlsx). L'agente le scopre e le usa automaticamente:

```
from agno.skills.agent_skills import Skills
from agno.skills.loaders.local import LocalSkills

skills = Skills(
    loaders=[LocalSkills(".agents/skills")]
)

agent = Agent(model=..., tools=tools, skills=skills, instructions=[...])
```

Nella istruzione è bene ricordare all'agente di recuperare le istruzioni della skill prima di usarla:

```
"Hai skill specializzate per documenti: docx, pdf, pptx, xlsx —
usa get_skill_instructions prima di lavorare su questi formati."
```

---

5. Agente Vision (immagini)

Per analizzare immagini si usa un modello multimodale (Pixtral):

```
from agno.media import Image

vision_agent = Agent(
    model=MistralChat(id="pixtral-large-latest", api_key="..."),
    tools=tools,
    instructions=[...],
)

image = Image(content=image_bytes)
response = await vision_agent.arun(
    input="Cosa vedi in questa immagine?",
    images=[image],
    session_id="utente-1",
)
```

---

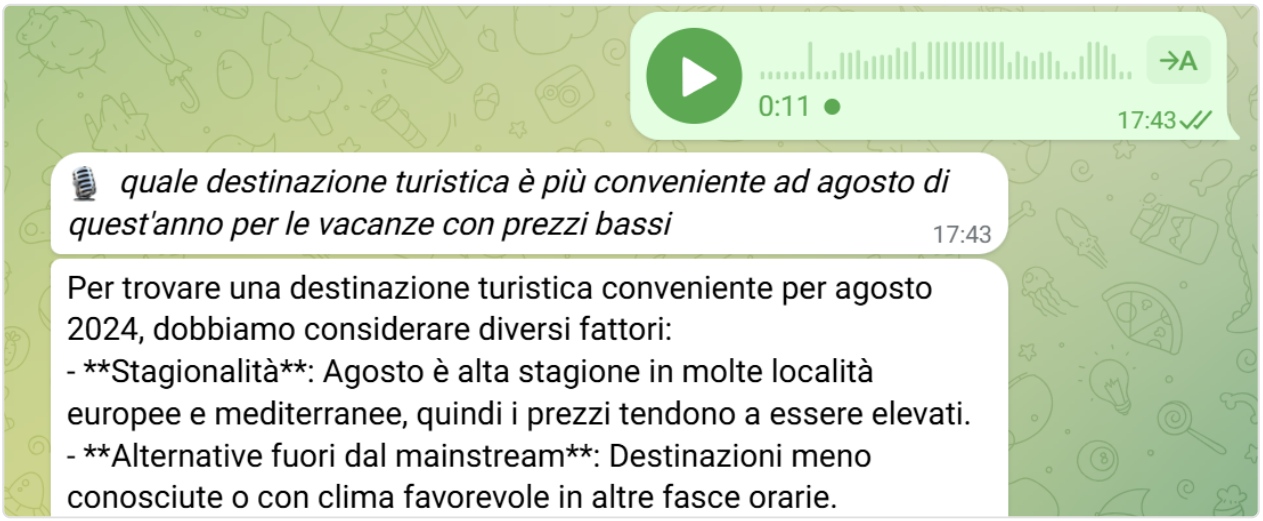
6. Integrazione con Telegram

L'agente viene esposto tramite FastAPI e riceve messaggi dal webhook di Telegram:

```python
# main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    await bot.register_webhook()
    yield
    await bot.deregister_webhook()

app = FastAPI(lifespan=lifespan)

@app.post("/webhook/telegram")
async def telegram_webhook(request: Request):
    update = await request.json()
    await bot.process_update(update)
```

```python
# handler.py
async def handle_message(update, context):
    response = await run_agent(
        user_input=update.message.text,
        session_id=str(update.effective_chat.id),
        user_id=str(update.effective_user.id),
    )
    await reply_long(update.message, response)
```

L'agente risponde direttamente in chat, anche a messaggi vocali (trascrizione automatica):



7. Retry e rate-limit

Le API LLM possono restituire errori 429. Un semaforo e un meccanismo di retry con backoff esponenziale rendono l'agente robusto:

```
import asyncio

_api_semaphore = asyncio.Semaphore(1)

async def _run_with_retry(coro_factory, max_retries=4):
    delays = [30, 60, 90, 120]
    async with _api_semaphore:
        for attempt in range(max_retries):
            try:
                response = await coro_factory()
                if response.status != RunStatus.error:
                    return response
            except Exception as exc:
                if "429" in str(exc) and attempt < max_retries - 1:
                    await asyncio.sleep(delays[attempt])
                    continue
                raise
```

Struttura del progetto

```
src/fbclaw/
├── main.py              # FastAPI app + lifespan
├── config.py            # Settings da .env (pydantic-settings)
├── bot/
│   ├── telegram_bot.py  # Registrazione webhook, processing update
│   ├── handler.py       # Ponte Telegram → run_agent()
│   └── utils.py         # reply_long() — split messaggi lunghi
└── agent/
    └── agent.py         # Agent, tools, memory, run_agent()
```

Riferimenti

- Documentazione Agno
- Modelli Mistral

- python-telegram-bot